C# do Iniciante ao Avançado

Guia prático de aprendizagem

Público-alvo: quem nunca programou ou quer consolidar os conhecimentos de C#
Objectivo: aprender a escrever, compreender, testar e organizar programas em C# com confiança, evoluindo dos primeiros passos até técnicas avançadas.

> Os exemplos usam a sintaxe moderna de C# e podem ser adaptados à versão do .NET instalada. Para começar, instale o SDK do .NET e use um editor como Visual Studio, Visual Studio Code ou Rider. Confirme a instalação com `dotnet --version`.

Índice

1. O que são C# e .NET
2. Criar e executar o primeiro programa
3. Variáveis, tipos e operadores
4. Decisões e ciclos
5. Métodos e âmbito
6. Arrays, listas e colecções
7. Texto, datas e enumerações
8. Classes e objectos
9. Programação orientada a objectos
10. Excepções e validação
11. Ficheiros e serialização JSON
12. LINQ
13. Genéricos
14. Delegados, eventos e expressões lambda
15. Programação assíncrona
16. Recursos avançados da linguagem
17. Testes, depuração e qualidade
18. Organização de aplicações e ferramentas .NET
19. Segurança e desempenho

20. Projectos práticos e plano de estudo
21. Glossário e referência rápida

---

1. O que são C# e .NET

C# (lê-se "C sharp") é uma linguagem de programação de propósito geral, fortemente tipada e orientada a objectos. É usada em aplicações Web, serviços, ferramentas de linha de comandos, aplicações empresariais, jogos com Unity, aplicações móveis e muito mais.

.NET é a plataforma que fornece o runtime, bibliotecas e ferramentas para compilar e executar aplicações C#. A linguagem e a plataforma são conceitos relacionados, mas diferentes: C# é a linguagem; .NET é uma das plataformas onde os programas C# correm.

Um programa passa, em termos gerais, por estas etapas: o código-fonte é compilado; o runtime .NET executa o resultado; as bibliotecas disponibilizam funcionalidades comuns, como operações com texto, ficheiros, rede e datas.

Vocabulário essencial

- **Código-fonte:** instruções escritas pelo programador.
- **Compilar:** verificar e transformar o código num formato executável pelo runtime.
- **Runtime:** ambiente que executa o programa.
- **Biblioteca:** código reutilizável que oferece funcionalidades.
- **IDE/editor:** ferramenta para escrever, executar e depurar código.
- **NuGet:** sistema de pacotes da plataforma .NET.

---

2. Criar e executar o primeiro programa

No terminal, crie uma aplicação de consola:

Bash

```bash
dotnet new console -n OlaCSharp
cd OlaCSharp
dotnet run
```

Em versões recentes, o ficheiro `Program.cs` pode conter apenas:

C#

```
Console.WriteLine("Olá, mundo!");
```

`Console.WriteLine` escreve uma linha no terminal. O ponto e vírgula termina a instrução. As chavetas `{}` delimitam blocos de código quando são necessários.

Uma versão explícita, útil para reconhecer a estrutura tradicional, é:

C#

```
using System;

namespace OlaCSharp;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("Olá, mundo!");
    }
}
```

`Main` é o ponto de entrada tradicional. Os programas modernos também podem usar **instruções de nível superior**, sem declarar explicitamente a classe `Program`.

Exercício

Altere a mensagem para incluir o seu nome e execute novamente com `dotnet run`.

---

3. Variáveis, tipos e operadores

Uma variável guarda um valor. Em C#, cada variável tem um tipo.

C#

```
string nome = "Inês";
int idade = 25;
double altura = 1.68;
decimal saldo = 125.50m;
bool estudante = true;
char inicial = 'I';
```

Tipos comuns:

Tipo	Utilização	Exemplo

`int`	inteiros	`42`
`long`	inteiros grandes	`9_000_000_000L`
`double`	valores reais aproximados	`3.14159`
`decimal`	valores decimais financeiros	`19.99m`
`bool`	verdadeiro/falso	`true`
`char`	um carácter	`'A'`
`string`	texto	`"Olá"`

Use `var` quando o compilador puder inferir claramente o tipo a partir do valor inicial:

```C#
var cidade = "Porto"; // O tipo inferido é string.
var quantidade = 3;   // O tipo inferido é int.
```

`var` não torna a variável dinamicamente tipada: depois de inferido, o tipo mantém-se. Por omissão, uma variável local tem de receber um valor antes de ser usada.

Constantes e valores opcionais

```C#
const int DiasDaSemana = 7;
int? temperatura = null; // int que também pode não ter valor.
```

O operador `??` fornece um valor alternativo:

```C#
string nomeApresentavel = nomePossivelmenteNulo ?? "Visitante";
```

Operadores

```C#
int soma = 4 + 3;
int diferenca = 4 - 3;
int produto = 4 * 3;
```

```
int quociente = 10 / 3; // 3: divisão inteira
int resto = 10 % 3;     // 1
```

Comparações: `==`, `!=`, `<`, `>`, `<=`, `>=`.
Lógicos: `&&` (E), `||` (OU), `!` (negação).

C#

```
bool podeEntrar = idade >= 18 && estudante;
```

Interpolação de texto torna as mensagens legíveis:

C#

```
Console.WriteLine($"{nome} tem {idade} anos.");
```

Conversões e entrada de dados

A conversão explícita pode perder informação; por exemplo, converter `double` para `int` descarta a parte decimal. Para entradas do utilizador, prefira métodos de análise seguros:

C#

```
Console.Write("Qual é a sua idade? ");
string? entrada = Console.ReadLine();

if (int.TryParse(entrada, out int idadeIntroduzida))
{
    Console.WriteLine($"Daqui a um ano terá {idadeIntroduzida + 1}.");
}
else
{
    Console.WriteLine("Introduza um número inteiro válido.");
}
```

`TryParse` evita usar excepções como mecanismo normal de validação.

---

4. Decisões e ciclos

`if`, `else if` e `else`

C#

```
if (idade < 0)
{
    Console.WriteLine("Idade inválida.");
}
else if (idade < 18)
{
    Console.WriteLine("Menor de idade.");
}
else
{
    Console.WriteLine("Maior de idade.");
}
```

switch

C#

```
string categoria = idade switch
{
    < 13 => "criança",
    < 18 => "adolescente",
    < 65 => "adulto",
    _ => "sénior"
};
```

A expressão `switch` devolve um valor. `_` representa o caso restante.

Ciclos

Use `for` quando o número de repetições ou o índice for importante:

C#

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

Use `while` quando a repetição depender de uma condição:

C#

```
string? comando;
while ((comando = Console.ReadLine()) != "sair")
{
```

```
    Console.WriteLine($"Comando: {comando}");
}
```

Use `foreach` para percorrer elementos:

C#

```
string[] cores = { "azul", "verde", "vermelho" };
foreach (string cor in cores)
{
    Console.WriteLine(cor);
}
```

`break` termina o ciclo; `continue` salta para a iteração seguinte. Evite ciclos infinitos acidentais e condições difíceis de compreender.

---

5. Métodos e âmbito

Um método reúne uma operação com nome e pode receber parâmetros e devolver um resultado.

C#

```
static int Somar(int a, int b)
{
    return a + b;
}

int total = Somar(4, 7);
```

`void` indica que o método não devolve um valor:

C#

```
static void MostrarBoasVindas(string nome)
{
    Console.WriteLine($"Bem-vindo/a, {nome}!");
}
```

Parâmetros opcionais podem ter valores predefinidos:

C#

```
static string Saudar(string nome, string idioma = "pt")
{
    return idioma == "pt" ? $"Olá, {nome}!" : $"Hello, {nome}!";
}
```

Âmbito é a região onde um nome é visível. Uma variável declarada dentro de um método, por exemplo, é local a esse método. Prefira métodos pequenos, com nomes claros e uma responsabilidade bem definida.

Parâmetros por referência

Parâmetros são passados por valor por omissão. `out` é útil em operações que produzem um resultado adicional, como `TryParse`. `ref` permite que o método altere a variável de quem o chamou; utilize-o apenas quando isso tornar a intenção mais clara.

---

6. Arrays, listas e colecções

Um **array** tem tamanho fixo:

C#

```
int[] notas = { 14, 17, 12 };
Console.WriteLine(notas[0]); // Índices começam em zero.
```

`List<T>` é uma lista dinâmica:

C#

```
using System.Collections.Generic;

var nomes = new List<string> { "Ana", "Rui" };
nomes.Add("Marta");
nomes.Remove("Rui");
Console.WriteLine(nomes.Count);
```

`Dictionary<TKey, TValue>` associa chaves a valores:

C#

```
var capitais = new Dictionary<string, string>
{
    ["Portugal"] = "Lisboa",
    ["França"] = "Paris"
};
```

```
if (capitais.TryGetValue("Portugal", out string? capital))
{
    Console.WriteLine(capital);
}
```

Outras estruturas úteis: `HashSet<T>` para valores únicos; `Queue<T>` para uma fila FIFO; `Stack<T>` para uma pilha LIFO. Escolha a colecção segundo as operações necessárias, não apenas por familiaridade.

---

7. Texto, datas e enumerações

As strings são imutáveis: operações que parecem alterar o texto produzem, na realidade, novas strings.

C#

```
string frase = "  C# é útil  ";
string limpa = frase.Trim();
string maiuscula = limpa.ToUpperInvariant();
bool contem = limpa.Contains("útil", StringComparison.OrdinalIgnoreCase);
```

Para construir texto repetidamente em ciclos, considere `StringBuilder` (`System.Text`). Para datas e horas, use `DateTime` ou, quando trabalha com instantes e fusos horários, `DateTimeOffset`.

C#

```
DateTime hoje = DateTime.Today;
DateTimeOffset agora = DateTimeOffset.UtcNow;
Console.WriteLine(agora.ToString("O"));
```

Use `TimeSpan` para durações. Para dados apresentados ao utilizador, formate segundo a cultura adequada; para guardar ou transmitir, prefira formatos invariantes e inequívocos.

Uma enumeração dá nomes a um conjunto de valores:

C#

```
enum EstadoEncomenda
{
    Pendente,
    Enviada,
    Entregue,
    Cancelada
```

```
}

EstadoEncomenda estado = EstadoEncomenda.Pendente;
```

8. Classes e objectos

Uma **classe** descreve dados e comportamentos; um **objecto** é uma instância dessa classe.

C#

```
public class Pessoa
{
    public string Nome { get; }
    public int Idade { get; private set; }

    public Pessoa(string nome, int idade)
    {
        if (string.IsNullOrWhiteSpace(nome))
            throw new ArgumentException("O nome é obrigatório.", nameof(nome));
        if (idade < 0)
            throw new ArgumentOutOfRangeException(nameof(idade));

        Nome = nome;
        Idade = idade;
    }

    public void FazerAniversario() => Idade++;
}

var pessoa = new Pessoa("Inês", 25);
pessoa.FazerAniversario();
Console.WriteLine(pessoa.Idade);
```

O **construtor** inicializa o objecto. Propriedades (`get` , `set`) expõem o estado. Encapsular o estado permite proteger invariantes e controlar alterações.

Tipos por valor e tipos por referência

Tipos primitivos como `int` e `bool` , bem como `struct` , são normalmente tipos por valor: uma atribuição copia o valor. Classes são tipos por referência: a variável guarda uma referência ao objecto. `null` significa ausência de uma referência/valor (em tipos anuláveis). Esta distinção é importante ao copiar, comparar e passar dados.

9. Programação orientada a objectos

Encapsulamento

Esconda detalhes internos e exponha operações seguras. Não torne tudo público por conveniência.

Herança

Uma classe pode especializar outra, mas a herança deve representar uma relação "é um". Prefira composição quando uma relação "tem um" descreve melhor o domínio.

C#

```
public abstract class Animal
{
    public abstract string EmitirSom();
}

public sealed class Gato : Animal
{
    public override string EmitirSom() => "Miau";
}
```

`abstract` impede a criação directa da classe e exige implementação dos membros abstractos. `sealed` impede novas subclasses.

Interfaces e polimorfismo

Uma interface define um contrato que tipos diferentes podem implementar:

C#

```
public interface IExportavel
{
    string Exportar();
}

public class Relatorio : IExportavel
{
    public string Exportar() => "Conteúdo do relatório";
}
```

O código que depende de `IExportavel` pode funcionar com diferentes implementações. Isto favorece testes e reduz acoplamento.

Composição

Uma classe pode usar outras classes para construir o seu comportamento. Em geral, a composição tende a ser mais flexível do que hierarquias de herança profundas.

---

10. Excepções e validação

Uma excepção representa uma situação excepcional, como um ficheiro inacessível. Trate apenas aquilo que consegue resolver ou contextualizar.

```C#
try
{
    string conteudo = File.ReadAllText("dados.txt");
    Console.WriteLine(conteudo);
}
catch (IOException ex)
{
    Console.Error.WriteLine($"Não foi possível ler o ficheiro: {ex.Message}");
}
```

Evite capturar `Exception` sem necessidade, ocultar erros com blocos `catch` vazios ou usar excepções para controlar situações normais. Lance excepções claras para argumentos inválidos; valide entradas junto à fronteira do sistema.

`finally` executa código de limpeza; para recursos descartáveis, prefira `using`:

```C#
using var leitor = new StreamReader("dados.txt");
string? linha = leitor.ReadLine();
```

---

11. Ficheiros e serialização JSON

`System.IO` contém APIs para ficheiros e directórios. Em aplicações reais, considere as permissões, caminhos, codificação e falhas de entrada/saída.

```C#
await File.WriteAllTextAsync("mensagem.txt", "Olá, ficheiro!");
string texto = await File.ReadAllTextAsync("mensagem.txt");
```

JSON é um formato comum para guardar e trocar dados. A biblioteca `System.Text.Json` permite serializar objectos:

C#

```
using System.Text.Json;

var produto = new Produto("Caderno", 3.50m);
string json = JsonSerializer.Serialize(produto);
Produto? copia = JsonSerializer.Deserialize<Produto>(json);

public record Produto(string Nome, decimal Preco);
```

Em aplicações robustas, trate dados inválidos, versões do formato e propriedades opcionais. Não desserialize dados não fiáveis para tipos perigosos nem assuma que a entrada é válida.

---

12. LINQ

LINQ (Language Integrated Query) permite filtrar, projectar, ordenar e agregar colecções.

C#

```
var numeros = new[] { 1, 2, 3, 4, 5, 6 };
var paresAoQuadrado = numeros
    .Where(n => n % 2 == 0)
    .Select(n => n * n)
    .OrderBy(n => n)
    .ToList();
```

Operações comuns: `Where` filtra; `Select` transforma; `OrderBy` ordena; `Any` verifica se existe algum elemento; `FirstOrDefault` obtém o primeiro ou o valor predefinido; `Count` conta; `Sum` soma; `GroupBy` agrupa.

Muitas consultas LINQ são avaliadas de forma diferida: só são executadas quando os resultados são percorridos ou materializados (`ToList` , `ToArray`). Tenha atenção a múltiplas enumerações, efeitos secundários e desempenho.

---

13. Genéricos

Os genéricos permitem escrever código reutilizável mantendo a segurança de tipos.

C#

```
static T Primeiro<T>(IReadOnlyList<T> elementos)
{
    if (elementos.Count == 0)
        throw new ArgumentException("A lista não pode estar vazia.", nameof(ele

    return elementos[0];
}

string primeiroNome = Primeiro(new[] { "Ana", "Rui" });
```

`T` é um parâmetro de tipo. Restrições (`where`) limitam os tipos permitidos:

```
C#

static T Criar<T>() where T : new() => new T();
```

As colecções .NET, como `List<T>` e `Dictionary<TKey, TValue>` , são exemplos de genéricos.

---

14. Delegados, eventos e expressões lambda

Um **delegado** representa uma referência a um método com uma assinatura específica. `Func<>` representa métodos que devolvem um valor; `Action<>` , métodos sem valor de retorno.

```
C#

Func<int, int, int> somar = (a, b) => a + b;
Console.WriteLine(somar(2, 3));
```

Uma expressão lambda é uma forma concisa de escrever uma função anónima, usada frequentemente com LINQ.

Eventos permitem notificar subscritores de que algo aconteceu:

```
C#

public class Temporizador
{
    public event EventHandler? Terminou;

    public void Concluir() => Terminou?.Invoke(this, EventArgs.Empty);
}
```

Ao subscrever eventos, considere remover a subscrição quando deixa de ser necessária, para evitar referências e retenções indesejadas.

---

15. Programação assíncrona

A programação assíncrona permite aguardar operações de entrada/saída sem bloquear desnecessariamente a thread que as iniciou.

C#

```
static async Task<string> ObterPaginaAsync(HttpClient cliente, string url)
{
    return await cliente.GetStringAsync(url);
}

using var cliente = new HttpClient();
string pagina = await ObterPaginaAsync(cliente, "https://example.com" );
```

- `Task` representa uma operação assíncrona sem resultado.
- `Task<T>` representa uma operação que produzirá um `T`.
- `async` permite usar `await` dentro de um método.
- `await` aguarda a operação sem bloquear a thread durante a espera.

Prefira propagar `async` / `await` até ao chamador. Evite `.Result` e `.Wait()` em código assíncrono, pois podem bloquear e causar problemas. Não use `async void`, excepto normalmente em handlers de eventos. Para cancelar, passe um `CancellationToken` às APIs que o suportam.

Para operações independentes, pode aguardar em conjunto com `Task.WhenAll`. Não crie tarefas em paralelo sem considerar limites, cancelamento e recursos partilhados.

---

16. Recursos avançados da linguagem

Records e imutabilidade

`record` é útil para modelos de dados cujo valor depende do conteúdo:

C#

```
public record Ponto(int X, int Y);
var origem = new Ponto(0, 0);
```

```
var deslocado = origem with { X = 5 };
```

Pattern matching

C#

```
static string Descrever(object? valor) => valor switch
{
    null => "nulo",
    int n when n > 0 => $"inteiro positivo: {n}",
    string { Length: 0 } => "texto vazio",
    string s => $"texto: {s}",
    _ => "outro tipo"
};
```

Tipos anuláveis de referência

Com `<Nullable>enable</Nullable>` no projecto, o compilador ajuda a identificar referências potencialmente nulas. Use `?` quando `null` for permitido e valide os dados externos. O operador `!` suprime avisos, mas não valida nem torna o valor seguro.

Iteradores

`yield return` produz elementos à medida que são pedidos:

C#

```
static IEnumerable<int> ContarAte(int limite)
{
    for (int i = 1; i <= limite; i++)
        yield return i;
}
```

`Span<T>` e `Memory<T>`

São abstracções para trabalhar com segmentos de memória de forma eficiente, frequentemente sem cópias. São temas avançados; use-os quando houver uma necessidade medida de desempenho e compreenda as restrições de vida útil e segurança.

Atributos e reflexão

Atributos acrescentam metadados a tipos e membros. Reflexão permite inspeccionar tipos em execução. São úteis em serialização, frameworks e ferramentas, mas aumentam complexidade e podem ter impacto no desempenho.

17. Testes, depuração e qualidade

Testes unitários

Um teste unitário verifica uma unidade pequena de comportamento. Os modelos mais comuns usam xUnit, NUnit ou MSTest; escolha um e siga a documentação da versão instalada.

Exemplo conceptual com xUnit:

C#

```
[Fact]
public void Somar_DevolveASoma()
{
    Assert.Equal(5, Somar(2, 3));
}
```

Um bom teste tem preparação, acção e verificação. Teste casos normais, limites e entradas inválidas. Evite testes dependentes da ordem, da rede ou do relógio quando não é necessário.

Depuração

Use pontos de interrupção, execução passo a passo, observação de variáveis e pilha de chamadas. Leia a mensagem e o tipo da excepção; não se limite a alterar código até o erro desaparecer.

Formatação e análise

Bash

```
dotnet format
dotnet build
dotnet test
```

Use nomes descritivos, métodos pequenos, princípios de responsabilidade única e análise estática quando disponível. Comentários devem explicar decisões e contexto, não repetir literalmente o código.

18. Organização de aplicações e ferramentas .NET

Comandos fundamentais:

```bash
dotnet new console -n MinhaApp
dotnet run
dotnet build
dotnet test
dotnet add package NomeDoPacote
dotnet list package
```

Uma solução pode conter vários projectos, por exemplo: aplicação, biblioteca de domínio e testes. Um ficheiro `.sln` agrupa projectos; o `.csproj` define as configurações do projecto.

Conceitos importantes ao desenvolver aplicações maiores:

- **Separação de responsabilidades:** apresentação, domínio e acesso a dados não devem ficar todos misturados.
- **Injecção de dependências:** fornecer dependências a uma classe em vez de esta as criar directamente; ajuda a substituir implementações e testar.
- **Configuração:** separar configurações de ambiente do código e não guardar segredos no repositório.
- **Registo (logging):** registar eventos úteis, sem expor palavras-passe ou dados sensíveis.
- **Controlo de versões:** usar Git para acompanhar alterações e colaborar.

Escolha a arquitectura proporcional ao problema. Uma aplicação pequena não precisa de camadas e padrões desnecessários.

---

19. Segurança e desempenho

Segurança

- Valide entradas externas e codifique correctamente dados apresentados em HTML, SQL ou outros contextos.
- Use consultas parametrizadas para bases de dados; nunca concatene dados do utilizador em SQL.
- Não inclua chaves, tokens ou palavras-passe no código nem no controlo de versões.
- Use HTTPS e bibliotecas mantidas; actualize dependências e avalie a sua origem.
- Aplique o princípio do menor privilégio e não registe informação sensível.
- Trate erros sem revelar detalhes internos a utilizadores finais.

Desempenho

Primeiro meça; depois optimize. Use um profiler e benchmarks representativos. Evite alocações repetidas em caminhos críticos, chamadas de I/O desnecessárias e consultas que enumeram dados várias vezes. Escolha estruturas de dados adequadas. Uma optimização só é útil se preservar correcção e legibilidade e se resolver um problema real.

---

20. Projectos práticos e plano de estudo

Projectos por nível

1. **Iniciante — Calculadora:** ler dois números, validar entradas e executar operações.
2. **Iniciante — Jogo de adivinhação:** gerar um número aleatório, usar ciclos e dar pistas.
3. **Intermédio — Gestor de tarefas:** classes, listas, menus, validação e persistência em JSON.
4. **Intermédio — Analisador de ficheiros:** ler CSV ou texto, filtrar dados com LINQ e gerar um resumo.
5. **Intermédio — Catálogo de biblioteca:** livros, empréstimos, regras de domínio e testes unitários.
6. **Avançado — Cliente de API:** chamadas HTTP assíncronas, cancelamento, tratamento de erros e desserialização.
7. **Avançado — Serviço Web:** API com ASP.NET Core, validação, logging, configuração e testes de integração.

Plano sugerido de oito semanas

- **Semana 1:** instalação, sintaxe, tipos, entrada e saída.
- **Semana 2:** condições, ciclos, métodos e exercícios pequenos.
- **Semana 3:** arrays, listas, dicionários, strings e datas.
- **Semana 4:** classes, objectos, encapsulamento e interfaces.
- **Semana 5:** excepções, ficheiros, JSON e LINQ.
- **Semana 6:** genéricos, delegados, eventos e testes.
- **Semana 7:** `async` / `await`, HTTP e organização de projectos.
- **Semana 8:** construir um projecto completo, rever código e documentar decisões.

A progressão depende do tempo disponível. A prática regular e a resolução de problemas

são mais importantes do que memorizar sintaxe.

Exercícios de revisão

1. Escreva um método que receba uma lista de inteiros e devolva o maior valor; defina o comportamento para lista vazia.
2. Conte quantas palavras existem numa frase, ignorando espaços repetidos.
3. Crie uma classe `ContaBancaria` que impeça saldos inválidos e permita depósitos e levantamentos.
4. Filtre com LINQ os números pares de uma lista e calcule a soma.
5. Guarde uma lista de tarefas em JSON e carregue-a novamente, tratando ficheiros inexistentes ou inválidos.
6. Escreva testes para os casos normais e extremos dos exercícios anteriores.

---

21. Glossário e referência rápida

- **API:** conjunto de operações disponibilizadas por uma biblioteca ou serviço.
- **Compilador:** programa que analisa e transforma código-fonte.
- **Classe:** definição de dados e comportamentos de objectos.
- **Interface:** contrato que declara operações que um tipo pode implementar.
- **Excepção:** mecanismo de representação de erros ou situações excepcionais.
- **Método:** bloco nomeado de código que executa uma operação.
- **NuGet:** gestor e repositório de pacotes .NET.
- **Objecto:** instância de uma classe ou de outro tipo de referência.
- **Runtime:** ambiente que executa o programa.
- **Tipo:** definição das operações e valores permitidos para um dado.

Comandos de bolso

Bash

```bash
dotnet --version              # versão do SDK
dotnet new console -n App     # criar aplicação de consola
cd App
dotnet run                    # executar
dotnet build                  # compilar
```

```
dotnet test                  # executar testes
dotnet format                # formatar código
```

Princípios para continuar a aprender

1. Escreva código e execute-o frequentemente.
2. Leia mensagens do compilador e investigue a causa dos erros.
3. Divida problemas grandes em passos pequenos.
4. Prefira soluções claras e simples antes de introduzir abstracções.
5. Teste casos extremos e dados inválidos.
6. Consulte a documentação oficial do C# e do .NET para confirmar detalhes dependentes da versão.
7. Reescreva projectos antigos com os conhecimentos novos: a revisão é uma parte essencial da aprendizagem.

Fim do guia. Este documento é uma base de estudo: para cada tema, a aprendizagem consolida-se com prática, leitura de código e projectos próprios.